

Regular Expressions

Regex | In Detail



`/(reg)ex/`

Mallikarjun | @CodeBustler



Regular Expressions

Regular Expressions (**RegeEx**) is used for **pattern matching** or string matching.

They are incredibly useful for tasks involving **search, validation, and manipulation of text**.

A regex consists of specific **characters & symbols** that define a search pattern.

In JS regEx starts & end with **/ slash**
e.g : /regEx/ 

RegEx | Quantifiers

Exprsn	Meaning
[]?	Occurs 0 or 1 time
[]+	Occurs 1 or more times
[]*	Occurs 0 or more times
[]{n}	Occurs n times
[]{n,}	Occurs n or more times
[]{y, z}	Occurs atleast y times, but less than z times
ab cd	match ab or cd

@CodeBustler

RegEx

Expression	Meaning
[abc]	a, b or c
[^abc]	anything except a,b,c
[a-z]	a to z
[A-Z]	A to Z
[a-m]	a to m
[a-zA-Z]	a to z, A to Z
[0-9]	0 to 9
● Full Stop	any character except newline

RegEx | Groups

Exprsn	Meaning
(abc)	capture group
(?:abc)	non-capturing group
(?=abc)	positive lookahead
(?!abc)	negative lookahead
(?<=abc)	positive lookbehind
(?<!abc)	negative lookahead
^abc\$ <i>Anchors</i>	starts and ends with "abc". ^ Starts \$ Ends

RegEx | Flags

Exprsn	It Allows
/... /g	Case Insensitive
/... /i	Global Match
/... /m	Allow Multiline
/... /u	Unicode
/... /y	Sticky
/... /s	DotAll

RegEx | MetaCharacters

Exprsn	Meaning
\d	Digit [0-9] @CodeBustler
\D	Not digit [^0-9]
\w	Word [a-zA-Z_0-9]
\W	Not word : \w 
\s	whitespace characters newline \n, tab \t, & \r
\S	Not whitespace chars 
\b	Word boundary \bword\b Matches "word"
\B	Not Word boundary

✓ RegExp Methods:

- `regex.exec(str)` : Searches for a match, returns array or null
- `regex.test(str)` : Tests for a match, returns true or false.

✓ String Methods: For RegExp

- `str.match(regex)`: Returns array / null
- `str.search(regex)`: Returns index / -1
- `str.replace(regex, replacement)`
- `str.split(regex)`: Splits string into array
 ↘ regex/separator

✓ RegExp Properties:

`const regex = /pattern/gim;`

- `regex.source` // Returns "pattern"
- `regex.global` // Returns true
- `regex.ignoreCase` // Returns true
- `regex.multiline` // Returns true
- `regex.lastIndex` // Index for next match
- `regex.flags`; // Returns "gim"